

CD UNIT 4

Design Issues of a Code Generator

The code generator is the final phase of the compiler, translating intermediate code into target machine code. Several design issues must be addressed to ensure the generated code is efficient, accurate, and suitable for the target architecture.

1. Input to the Code Generator

- The code generator takes **intermediate representation (IR)** as input, created by the front end of the compiler.
- This IR must be free of errors, as all syntax and semantic checks are performed earlier.
- The input includes:
 - Linear representation: Postfix or three-address code (e.g., quadruples, triples).
 - Virtual machine representation: Stack machine code.
 - Graphical representation: Syntax trees, DAGs.
- **Symbol table** provides information about the runtime addresses of variables, constants, and temporary results.

2. Target Program

- The target program could be one of the following:
 1. **Absolute Machine Code**: A direct machine code that can be executed immediately.
 2. **Relocatable Machine Code**: Allows subprograms to be compiled separately.
 3. **Assembly Code**: Simplifies debugging and further processing.
- The **Instruction Set Architecture (ISA)** (RISC, CISC, stack-based) influences code generator design, as it determines the available instructions and addressing modes.

3. Memory Management

- The code generator maps variable names to their runtime memory addresses using the symbol table.
- **Labels** in IR are converted into actual memory addresses for jump instructions.
- Example:
 - A jump like `goto L1` in IR must translate into the correct address of the instruction labeled `L1` in the target code.

4. Instruction Selection

- The code generator selects the most suitable machine instructions to implement the IR operations.

- The **quality of the code** is judged by:
 - **Efficiency:** Instructions should execute quickly.
 - **Size:** The generated code should use minimal memory.
- **Machine idioms** (specific instruction sequences) are often used to enhance efficiency.

5. Register Allocation

- Registers are faster than memory, so frequent values should reside in registers.
- This involves two subproblems:
 1. **Register Allocation:** Deciding which variables should be stored in registers.
 2. **Register Assignment:** Selecting specific registers for these variables.
- Certain architectures require specific registers or register pairs for particular instructions, complicating allocation.

6. Evaluation Order

- The order of computation affects the use of registers and memory.
- Efficient order minimizes:
 - Number of registers needed for temporary results.
 - Movement between memory and registers.
- Example: Rearranging computations to delay certain calculations can reduce the number of instructions.

Peephole Optimization

Peephole optimization is a local optimization technique that focuses on improving a small sequence of target code instructions (known as a **peephole**) by replacing it with a shorter, faster, or more efficient version. The technique involves scanning the code through a "sliding window" and making changes wherever patterns for improvement are detected.

This process is applied after the initial code generation phase and can lead to significant improvements in the performance and size of the final code. Peephole optimization is effective because it targets redundant or inefficient instructions generated during the compilation process.

Techniques of Peephole Optimization

1. **Redundant Loads and Stores**
 - Removes unnecessary load/store instructions where the same value is loaded or stored repeatedly.

Example:

```
MOV R0, a
```

```
MOV a, R0
```

- The second instruction can be eliminated as `R0` already holds the value of `a`.

2. Unreachable Code Removal

- Eliminates code that can never be executed due to logical flow or conditional jumps.

Example:

```
JMP L2
```

```
L1: ADD R1, R2, R3 // This code is unreachable
```

```
L2: SUB R4, R5, R6
```

- The instruction at `L1` can be removed.

3. Flow-of-Control Optimization

- Simplifies sequences of jumps or branches to improve control flow.

Example:

```
JMP L1
```

```
L1: JMP L2
```

This can be simplified to:

```
JMP L2
```

-

4. Algebraic Simplifications

- Simplifies arithmetic expressions using mathematical identities.

Example:

```
x = x + 0 // Can be eliminated
```

```
x = x * 1 // Can be eliminated
```

-

5. Reduction in Strength

- Replaces expensive operations with cheaper equivalents.

Example:

```
x = x * 2 // Replace with
```

```
x = x + x // Addition is faster than multiplication
```

Benefits of Peephole Optimization

- **Improves code efficiency** by reducing execution time and memory usage.
- **Minimizes code size**, leading to faster execution.
- **Easy to implement**, as it targets small and local sections of code.

Data Structures Used to Implement a Symbol Table

A **symbol table** is a data structure used in compilers to store information about program entities such as variables, functions, and objects. It plays a crucial role in semantic analysis, code generation, and optimization by mapping names to their corresponding attributes (type, scope, memory location, etc.).

Several data structures can be used to implement a symbol table, each with its advantages and trade-offs in terms of speed, memory efficiency, and complexity.

1. Linear List

- **Description:**
 - A linear list is a simple data structure where names and their attributes are stored sequentially in an array or linked list.
 - New entries are added to the end of the list.
- **Operations:**
 - **Insertion:** Add the new name at the end.
 - **Search:** Scan the entire list to find a name.
- **Advantages:**
 - Simple to implement.
 - Suitable for small programs.
- **Disadvantages:**
 - Search time is linear ($O(n)$), which makes it inefficient for large symbol tables.
 - Insertions are costly in terms of time for large tables.

Example:

A table with variables {x, y, z}:

```
Index: 0    1    2
```

Name : x y z

2. Self-Organizing List

- **Description:**
 - An improvement on the linear list where frequently accessed entries are moved closer to the beginning to reduce average search time.
 - This is achieved by maintaining a "linker" field that rearranges the order of entries dynamically.
 - **Advantages:**
 - Reduces the average search time for frequently accessed names.
 - **Disadvantages:**
 - Still has a worst-case time complexity of $O(n)$ for search and insertions.
-

3. Binary Search Tree (BST)

- **Description:**
 - A tree structure where each node contains a name, and left and right pointers.
 - Names are stored in sorted order: the left child stores smaller names, and the right child stores larger names.
 - Searches, insertions, and deletions are performed using the properties of a BST.
- **Operations:**
 - **Insertion:** Follow BST rules to place the new name.
 - **Search:** Traverse the tree based on comparisons.
- **Advantages:**
 - Efficient for large tables with a time complexity of $O(\log n)$ for balanced trees.
 - Naturally supports scoping by associating subprograms with subtrees.
- **Disadvantages:**
 - Performance degrades to $O(n)$ for unbalanced trees.

Example:



4. Hash Table

- **Description:**
 - Uses a hash function to map names to specific locations (buckets) in an array.
 - If multiple names map to the same location, a collision occurs and is resolved using techniques like chaining or open addressing.
- **Operations:**
 - **Insertion/Search:** Compute the hash and insert/find the name in the corresponding bucket.
- **Advantages:**
 - Fast search, insertion, and deletion with average time complexity of $O(1)$.
 - Efficient for large symbol tables.
- **Disadvantages:**
 - Requires careful design of the hash function to minimize collisions.
 - Collisions increase search time.

Example:

Hash Table:

Index	Bucket
0	[a]
1	[x, y]
2	[z]

Comparison

Data Structure	Search Time	Insertion Time	Space Efficiency	Ease of Implementation
Linear List	$O(n)$	$O(1)$	High	Easy
Self-Organizing List	$O(n)$ (on avg.)	$O(1)$	High	Moderate
Binary Search Tree	$O(\log n)$	$O(\log n)$	Moderate	Moderate
Hash Table	$O(1)$ (avg.)	$O(1)$	Moderate	Complex

Next-Use Information

- If the name in a register is no longer needed, then we remove the name from the register and the register can be used to store some other names.
- Next use information is used in code generation and register allocation.

The use of a name in a three-address statement is defined as

Let the TAC statement “i” assigns a value to the element x,

- If TAC statement “j” has x as an operand and control can flow from statement “i” to “j” along the path there is no intervening assignments to x, then the statement “j” uses the value of x computed at statement “i”.
- The Next use Computation algorithm assumes that on exit from the basic block,
 1. All temporaries are considered non-live
 2. All variables defined in the source program (and non-temps) are live
 3. Each procedure/function call is assumed to start a basic block

Algorithm to determine Liveliness and Next-use information

Input : Basic block B of three-address statements

Output : At each statement i: $x = y \text{ op } z$, we attach to i the liveliness and next-uses of x, y and z.

Method : We start at the last statement of B and scan backwards.

1. Attach to statement i the information currently found in the symbol table regarding the next-use and liveliness of x, y and z.
2. In the symbol table, set x to “not live” and “no next use”.
3. In the symbol table, set y and z to “live”, and next-uses of y and z to i.

Example: Consider the following TAC sequence in a Basic Block

[NNU – No Next Use; NU – Next Use, NLive – Not Live]

(1)	prod := 0	{prod- [Live, NNU] }
(2)	i := 1	{ i – [Live, NU – (3)] }
(3)	t1 := 4* i	{ t1 – [NLive, NU – (4), i-[Live, NNU]] }
(4)	t2 := a[t1]	{ t2 – [NLive, NNU], t1 – [NLive, NNU], a – [Live, NNU] }

Symbol Table

A **symbol table** is an essential data structure used in compilers to store and manage information about the names (symbols) used in a program. These symbols may include variable names, function names, object names, constants, and other identifiers. The symbol table is built during the compilation process and is used throughout the analysis and synthesis phases of the compiler.

Purpose of a Symbol Table

- **Store Names:** Keeps all names of variables, functions, objects, and other entities in a structured format.
 - **Type Verification:** Verifies the type consistency of variables during semantic analysis.
 - **Scope Management:** Determines the scope (visibility) of each name in the program.
 - **Run-Time Information:** Provides information like storage class, memory offset, and dimensions for code generation.
 - **Error Detection:** Helps in identifying undeclared variables or multiple declarations.
-

Components of a Symbol Table

The symbol table consists of various attributes for each symbol to provide detailed information. These attributes are as follows:

1. **Name:**
 - The name of the symbol as used in the source program (e.g., variable name, function name).
 - Acts as a key for lookups in the symbol table.
2. **Type:**
 - The data type of the symbol (e.g., `int`, `float`, `array`, `struct`).
 - For functions, it includes the return type and the parameter types.
3. **Scope:**
 - Indicates the region of the program where the symbol is valid.
 - Used to manage global, local, and nested scopes.
4. **Storage Class:**
 - Specifies how the symbol is stored in memory (e.g., static, auto, register).
5. **Memory Location/Offset:**
 - The address or memory location where the symbol's value is stored during execution.
6. **Dimension (if applicable):**
 - For arrays and multidimensional variables, this stores the size of each dimension.
7. **Parameter List (for functions):**
 - Contains details about the formal parameters, including names, types, and passing modes (by-value or by-reference).
8. **Value (if constant):**
 - For constants, the symbol table stores the actual value.
9. **Linkage Information:**
 - Indicates external or internal linkage for symbols across modules in programs with multiple files.

The DAG Representation for Basic Blocks

- A DAG for a basic block is a directed acyclic graph with the following labels on nodes
 1. Leaves are labeled by unique identifiers, either variable names or constants.
 2. Interior nodes are labeled by an operator symbol.
 3. Nodes are also optionally given a sequence of identifiers for labels to store the computed values.
- DAGs are useful data structures for implementing transformations on basic blocks.
- It gives a picture of how the value computed by a statement is used in subsequent statements.
- It provides a good way of determining common sub - expressions.

Application of DAG

1. We can automatically detect common sub expressions.
2. We can determine which identifiers have their values used in the block.
3. We can determine which statements compute values that could be used outside the block.

Basic Block Construction Algorithm

Algorithm : Partition into basic blocks

Input : A sequence of three-address statements

Output : A list of basic blocks with each three-address statement in exactly one block

Steps :

1. First determine the set of leaders, the first statements of basic blocks.
The rules for determining the Leaders are,
 - a. The first statement is a leader.
 - b. Any statement that is the target of a conditional or unconditional goto is a leader.
 - c. Any statement that immediately follows a conditional or unconditional goto statement is a leader.
2. For each leader, its basic block consists of the leader and all statements up to but not including the next leader or the end of the program.

THE PRINCIPLE SOURCES OF OPTIMIZATION

- A transformation of a program is called local if it can be performed by looking only at the statements in a basic block; otherwise, it is called global.
- Many transformations can be performed at both the local and global levels. Local transformations are usually performed first.

Function-Preserving Transformations / Semantics Preserving Transformations

1. Common Subexpression Elimination

- **Goal:** Avoid redundant calculations of the same expression.

Example:

```
t1 = 4 * i
```

```
t2 = a[t1]
```

```
t3 = 4 * i
```

After elimination:

```
t1 = 4 * i
```

```
t2 = a[t1]
```

2. Copy Propagation

- **Goal:** Replace a variable with its value where possible, making some code redundant.

Example:

Before:

```
x = pi
```

```
A = x * r * r
```

After propagation:

```
A = pi * r * r
```

3. Dead Code Elimination

- **Goal:** Remove code that does not affect the program's outcome.

Example:

```
i = 0
```

```
if (i == 1) {
```

```
    a = b + 5
```

```
}
```

- The condition `i == 1` is never true, so the block can be eliminated.
-

4. Constant Folding

- **Goal:** Precompute constant expressions during compilation.

Example:

Before:

```
a = 3.14 / 2
```

After folding:

```
a = 1.57
```

CD UNIT 2

YACC: Yet Another Compiler-Compiler

YACC is a tool used for building **LALR(1) parsers** (Lookahead Left-to-Right parsing with 1 lookahead token). It automates the process of constructing a parser for a language from its grammar rules, making it easier to develop language parsers for compilers and interpreters.

What YACC Does

1. Takes a grammar specification as input.
2. Generates a parser in the C programming language.
3. The parser checks whether an input string conforms to the grammar and constructs parse trees if needed.

YACC is typically used with **Lex**, a lexical analyzer generator. While Lex tokenizes the input, YACC uses the tokens to parse and analyze syntax.

Structure of a YACC Input File

A YACC file is divided into three sections:

1. **Definitions Section:**
 - Used to declare tokens and include any external code.

Example:

```
%{  
#include <stdio.h>  
%}  
%token NUMBER
```

2. Rules Section:

- Specifies the grammar of the language using production rules in a modified BNF (Backus-Naur Form).
- Each production rule can include **actions** (C code) enclosed in `{ }` to define what happens when that rule is matched.

Example:

```
expr : expr '+' term { printf("Addition\n"); }  
      | term;
```

3. Auxiliary Routines Section:

- Contains C code for functions used in the rules section.
- It may also include the `main()` function if the parser will run as a standalone program.

Example:

```
int main() {  
    return yyparse();  
}
```

Key Components

1. Tokens:

- Represent the basic building blocks of the language (e.g., keywords, operators).
- Declared using `%token`.

2. Grammar Rules:

- Define how tokens can be combined to form valid constructs.

3. Actions:

- Embedded C code that executes when a rule is matched.

4. Parser Function (`yyparse`):

- Automatically generated by YACC. It processes the input tokens and applies the grammar rules.
-

Output of YACC

- `y.tab.c`: The generated C code for the parser.
-

Example of a YACC Grammar

This example parses arithmetic expressions like `3 + 5`.

```
%{
#include <stdio.h>
%}

%token NUMBER

%%

expr : expr '+' term { printf("Addition\n"); }
    | term;

term : NUMBER { printf("Number: %d\n", $1); };

%%

int main() {
    yyparse();
}

int yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
    return 0;
}
```

Key Features of Yacc

1. Grammar-Based Parsing

Yacc uses context-free grammar (CFG) rules to define the structure of the input language. It supports left-recursive and right-recursive grammar.

2. Automatic Parser Generation

Generates a bottom-up parser (LALR(1) parser) based on the specified grammar. Automates the process of creating a syntax tree from tokens produced by a lexical analyzer.

3. Token Management

Works with token streams, often provided by a lexical analyzer like Lex. Tokens are defined using `%token` in Yacc or imported from Lex.

.

4. Operator Precedence and Associativity

Allows specification of operator precedence and associativity directly in the grammar using %left, %right, and %nonassoc.

5. Resolves ambiguities in arithmetic and logical expressions.

```
%left '+' '-'
```

```
%left '*' '/'
```

6. Error Detection and Recovery

Built-in error detection using the reserved token error.

Provides mechanisms for error recovery, allowing the parser to continue parsing after encountering an error.

yacc

```
stmt: error ';' { printf("Error recovered\n"); }
```

7. Modularity

Combines well with other tools (e.g., Lex for lexical analysis).

Supports splitting grammar definitions and actions into manageable sections.

8. C Language Integration

Generates parsers in C, enabling easy integration with C-based projects.

Allows embedding of C code directly within the grammar file for custom logic and data manipulation.

9. Multiple Rule Alternatives

Supports multiple alternative forms for a single rule using the | operator.

yacc

Copy code

```
expr: expr '+' expr  
    | expr '-' expr  
    | NUM;
```

Error Recovery in Yacc

Yacc provides mechanisms to detect and recover from errors in input parsing. It uses the special reserved token error to handle syntax errors. When the parser encounters an invalid input, it invokes error recovery rules defined in the grammar. This helps the parser continue processing input beyond the error, instead of terminating immediately.

Yacc for a desk calculator with error recovery

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int yylex();
void yyerror(const char *s);
%}
```

```
%token NUM
%left '+' '-'
%left '*' '/'
%left '(' ')'
```

```
%%
```

```
calclist:
```

```
    /* empty */
    | calclist expr '\n' { printf("Result: %d\n", $2); }
    | calclist error '\n' { yyerror("Invalid expression. Skipping input."); yyerrok; }
    ;
```

```
expr: expr '+' expr    { $$ = $1 + $3; }
    | expr '-' expr    { $$ = $1 - $3; }
    | expr '*' expr    { $$ = $1 * $3; }
    | expr '/' expr    {
        if ($3 == 0) {
            yyerror("Division by zero error.");
            $$ = 0;
        } else {
            $$ = $1 / $3;
        }
    }
    | '(' expr ')'      { $$ = $2; }
    | NUM               { $$ = $1; }
    ;
```

```
%%
```

```
int main() {
    printf("Enter expressions (Ctrl+D to exit):\n");
    yyparse();
    return 0;
}
```

```
void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}
```

Features of Recursive Descent Parser

A recursive descent parser is a top-down parsing technique that reads input left-to-right and constructs the parse tree starting from the root. It uses a collection of mutually recursive procedures, each corresponding to a non-terminal in the grammar.

Key Features

1. Top-Down Parsing
 - The parser starts at the root of the parse tree (the start symbol) and works its way down to the leaves (tokens in the input string).
 - Constructs the parse tree in a preorder traversal.
2. Recursive Procedures
 - Each non-terminal in the grammar corresponds to a recursive procedure.
 - The procedures call one another to match input tokens against grammar rules.
3. Context-Free Grammar Parsing
 - Designed to work with context-free grammars (CFGs).
 - Handles grammars without left recursion.
4. Backtracking
 - Some versions of recursive descent parsers require backtracking when a production rule fails.
5. Simple Implementation
 - Easy to implement manually.
6. Predictive Parsing Variant
 - A specific form, called predictive parsing, eliminates backtracking by using lookahead tokens to decide the next production.
 - Uses a FIRST and FOLLOW set to determine valid grammar rules.

CD UNIT 3

BACKPATCHING

- A key problem when generating code for Boolean expression and flow-of-control statements is that of matching a jump instruction with the target of the jump.
- TAC for Boolean expressions and flow-of-control statements are generated using two passes
 - Pass 1 – Generate the TAC instruction with the target of the jumps are unspecified.
 - Pass 2 – Identifying and filling up the labels at appropriate jump statements.

- To generate TAC instructions in a single pass, a compiler may not know the labels that control must goto at the time of goto statements are generated.
- Backpatching can be used to generate code for Boolean expressions and flow-of-control statements in one pass.
- It maintains a list of TAC statements that needed to be completed with the same label. All of the jumps on a list have the same target label.

Attributes:

B.trueList – will be list of jump instructions into which control goes if B is true.

B.falseList – will be list of jump instructions into which control goes if B is false.

S.nextList – denoting a list of jumps to the instruction immediately following the code for S

Routines:

Makelist (i) – creates a new list containing only i.

Merge (p1,p2) – Concatenates the ls its pointed to by p1 with p2

Backpatch (p,i) – inserts i as target label for each of the instructions on the list pointed to by p.

Bottom Up parser:

A bottom-up parsing corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top).

- Bottom-up parsers generally operate by choosing, on the basis of the next input symbol (look-ahead symbol) and the contents of the stack, whether to shift the next input onto the stack, or to reduce some symbols at the top of the stack.
- A reduce step takes a production body at the top of the stack and replaces it by the head of the production

How Bottom-Up Parsing Works

1. **Shift:** Move the next input symbol onto the stack.
2. **Reduce:** Replace a sequence of symbols (matching the right-hand side of a grammar rule) at the top of the stack with the left-hand side of that rule.
3. **Accept:** If the entire input has been successfully parsed, announce completion.
4. **Error:** If the parser cannot continue due to mismatched input, an error is reported.

Types of Bottom-Up Parsers

1. **Shift-Reduce Parser:**
 - Uses a stack to manage symbols and grammar rules.
 - Performs **Shift** and **Reduce** actions to build the parse tree.
2. **Operator Precedence Parser:**
 - Handles grammars with precedence relations ($<$, $>$, $=$) between operators.

3. **LR Parsers** (most powerful):
 - **SLR (Simple LR)**: Handles smaller grammars with fewer states.
 - **CLR (Canonical LR)**: Handles complete LR(1) grammars, has more states.
 - **LALR (Look-Ahead LR)**: Combines efficiency of SLR with power of CLR.

Recovery modes:

1) Panic Mode Recovery

- **How it works:**

When an error is found, the parser **throws away input symbols** one by one until it finds a **specific symbol** (like `;` or `}`) that indicates it can continue.
 - **Example:**

If an error occurs in `int x =` `;`, the parser might skip to the next semicolon `;` to recover.
-

2) Phrase Level Recovery

- **How it works:**

The parser **fixes small parts** of the input to make it valid and then continues parsing. Example fixes include adding a missing character or replacing a wrong one.
 - **Example:**

For `int x =` `;`, the parser might correct it to `int x = 0;` and move on.
-

3) Error Productions

- **How it works:**

The grammar is extended to include **common mistakes** that programmers might make. If one of these mistakes is detected, the parser can handle it gracefully.
 - **Example:**

For a missing semicolon error, the parser might recognize `int x = 5` (without `;`) as a common mistake and report it as "missing semicolon."
-

4) Global Corrections

- **How it works:**
The parser tries to make **minimal changes** to the input to turn it into a valid program. It uses algorithms to find the **best way to fix errors** with the least effort.
 - **Example:**
If the input is `int x 5;`, the parser might fix it to `int x = 5;` by adding `=`.
-

Summary Table

Method	What it does	Best for	Drawback
Panic Mode	Skips input until a specific symbol	Simple and avoiding infinite loops	May skip valid code too
Phrase Level	Fixes small mistakes locally	Avoiding large skips	Hard to handle deeper errors
Error Productions	Recognizes common errors in grammar	Reporting clear error messages	Makes grammar more complex
Global Corrections	Makes minimal changes to fix input	Producing accurate corrections	Expensive and complex to compute

This simplifies how parsers recover from errors while explaining the pros and cons of each strategy.

UNIT 1:

Impacts on Compilers

- Compilers can help promote the use of high-level languages by minimizing the execution overhead of the programs written in these languages.
- Compilers are also critical in making high-performance computer architectures effective on users' applications.
- The performance of a computer system is so dependent on compiler technology that compilers are used as a tool in evaluating architectural concepts before a computer is built.
- Compiler writing is challenging. A compiler by itself is a large program.

- A compiler must translate correctly the potentially infinite set of programs that could be written in the source language.
- The problem of generating the optimal target code from a source program is undecidable.

COMPILER CONSTRUCTION / WRITING TOOLS

1. Scanner Generators – produce lexical analysers from a regular expression description of the tokens of a language. Example: LEX tool.
2. Parser Generators – automatically produce syntax analyzer from a grammatical description of a programming language.
3. Syntax-Directed translation Engines – produce a collection of routines for traversing a parse tree and generating intermediate code.
4. Code Generator Generators – produce a code generation from a collection of rules for translating each operation of the intermediate language into machine language for the target machine.
5. Data Flow Analysis Engine – facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data flow analysis is a key part of code optimization.
6. Compiler Construction Toolkit – provide an integrated set of routines for constructing various phases of a compiler.

The features of the LEX tool

- Specification Language: LEX is a specialized language for specifying lexical analyzers using regular expressions to describe token patterns.
- Transition Diagram Generation: LEX translates input patterns into a transition diagram, simulating them using C code generated in lex.yy.c.
- Three-Part Program Structure:
 - Declarations Section: Variables, constants, and regular definitions.
 - Translation Rules Section: Rules pairing patterns with actions (written in C) to process lexemes.
 - Auxiliary Procedures Section: Optional procedures to support translation actions.
- Pattern Matching Primitives: Supports meta-characters such as *, +, |, [], ^, and \$ for flexible pattern definition.
- Execution Workflow: The .l file is passed to the LEX compiler to generate lex.yy.c, which is then compiled with a C compiler to produce an executable lexical analyzer.

- **Predefined Variables:**Includes variables like yytext (matched string), yyleng (length of the matched string), and yyin/yyout (input/output files).
- **Conflict Resolution:**Resolves pattern conflicts by preferring the longest match and, if lengths are equal, the first-listed pattern.
- **Efficiency in Token Recognition:**Utilizes efficient algorithms to generate and execute finite automata for token recognition, improving processing speed.
- **Integration with C:**Actions and additional logic are written in C, allowing extensive flexibility and control.
- **Portability and Modularity:**Generates modular code that can be used across different platforms and integrated into larger compilers.

Bootstrapping

Bootstrapping is a process in which simple language is used to translate more complicated program which in turn may handle for more complicated program. This complicated program can further handle even more complicated program and so on. Applying this technique leads to a self-hosting compiler.

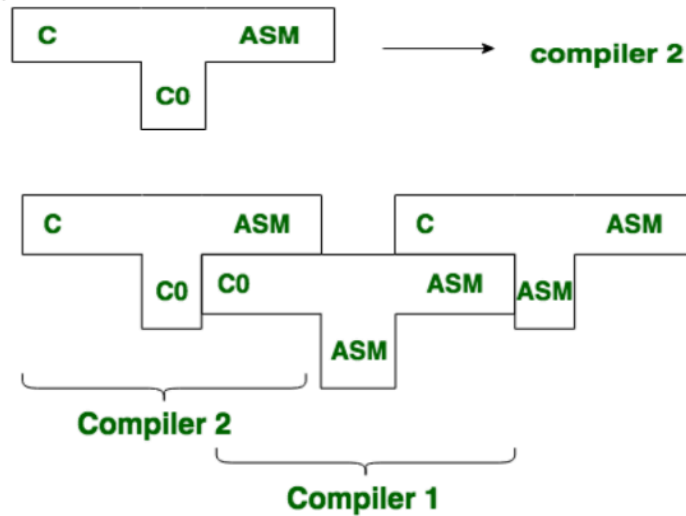
Compiler which takes C language and generates an assembly language as an output with the availability of a machine of assembly language.

Step-1: First we write a compiler for a small of C in assembly language.



Step-2: Then using with small subset of C i.e. C0, for the source language c the compiler is written.

Step-3: Finally we compile the second compiler. using compiler 1 the compiler 2 is compiled.



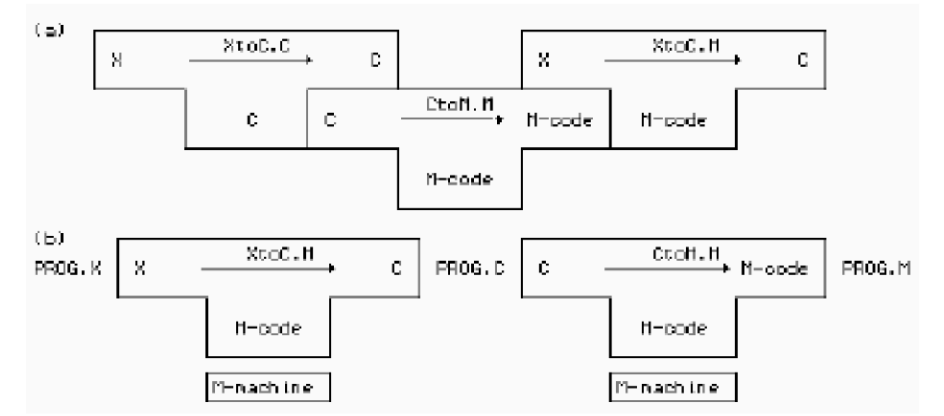
Step-4: Thus we get a compiler written in ASM which compiles C and generates code in ASM.

Porting

The process of modifying an existing compiler to work on a new machine is often known as porting the compiler.

Simplified Example of Porting

1. **Scenario:**
You have a compiler called **X-to-C**, written in **C**, that runs on **Machine A**. This compiler translates programs written in **X language** into C code.
2. **Goal:**
You want this compiler to run on **Machine B**.
3. **Steps:**
 - Take the source code of the **X-to-C compiler** from **Machine A**.
 - Move the source code to **Machine B**.
 - Use an existing **C compiler on Machine B** to compile the **X-to-C source code**.
 - Now, you have an **X-to-C compiler** that runs on **Machine B**.



Token,lexeme and pattern

1. Token: A **token** is a sequence of characters that forms a single logical unit in the source code. The lexical analyzer identifies tokens during the scanning process.

Examples from the Document:

- **Keywords:** `if`, `int`, `else`
- **Operators:** `+`, `-`, `=`
- **Identifiers:** `x`, `total`
- **Constants:** `3.14`, `100`

Example from Code:

```
int x = 10;
```

- **Tokens identified:**
 - `int` → **Keyword**
 - `x` → **Identifier**
 - `=` → **Operator**
 - `10` → **Constant**

2. Lexeme: A **lexeme** is the sequence of characters in the source program that matches a pattern for a token. It is the **instance** or actual string found in the code that represents a token.

Examples from the Document:

- For the token **Keyword**: `int`, `if` are lexemes.
- For the token **Identifier**: `x`, `y`, `area` are lexemes.

Example from Code:

```
int x = 10;
```

- **Lexemes extracted:**
 - `int`
 - `x`
 - `=`
 - `10`
-

3. Pattern: A **pattern** is a rule or description that defines what sequences of characters are recognized as a particular token. The lexical analyzer uses regular expressions to describe these patterns.

Examples of Patterns from the Document:

- **Keywords:** Specific words like `if`, `else`, `int`
- **Identifiers:** `[A-Za-z][A-Za-z0-9]*`
- **Constants:** `[0-9]+(\.[0-9]+)?`

Example from Code:

For the token **Identifier**, the pattern `[A-Za-z][A-Za-z0-9]*` matches:

- `x`
- `variable1`

For the token **Number**, the pattern `[0-9]+` matches:

- `10`
 - `123`
-

Relationship Between Token, Pattern, and Lexeme (Based on Document)

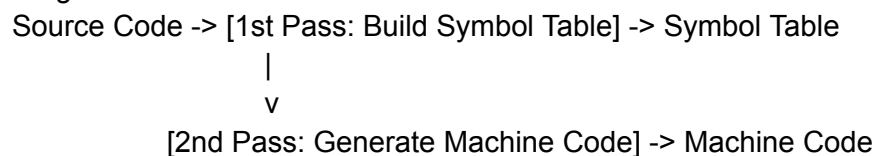
Token	Pattern	Lexeme
Keyword	Specific words (<code>int</code> , <code>if</code>)	<code>int</code>
Identifier	<code>[A-Za-z][A-Za-z0-9]*</code>	<code>x</code>
Operator	Symbols (<code>+</code> , <code>=</code> , <code>*</code>)	<code>=</code>
Constant	<code>[0-9]+(\.[0-9]+)?</code>	<code>10</code>

Organizational Issue of 2-Pass Assembler with Diagram

A 2-pass assembler translates assembly code into machine code in two phases.

- First Pass: The assembler scans the source code to build a symbol table. This table stores information like the addresses of labels (like `start` or `loop`).
- Second Pass: Using the symbol table, the assembler replaces labels with actual memory addresses and generates the machine code.

Diagram:



- Symbol Table contains labels and their corresponding addresses.
- The assembler first collects all labels and addresses, and then in the second pass, it translates the instructions.

Example:

Consider the following code:

```
start: MOV A, 5
      ADD B
loop:  SUB C
      JMP start
```

- First pass builds the symbol table:

`start` -> Address 100

`loop` -> Address 200

- Second pass generates machine code using the symbol table.

Input Buffering Pseudocode

Input Buffering is used to efficiently read input data for processes like lexical analysis. It reduces the number of reads from the input stream.

Essential Character: It is the character that is necessary to continue processing the input (e.g., the next character in a token).

Pseudocode for Input Buffering:

```
buffer = read_input() // Read input into a buffer
current_position = 0  // Track current position in the buffer
```

```
function get_next_token():
    while current_position < len(buffer):
        char = buffer[current_position]
        if char is a valid token character:
            return char
        current_position += 1
    return EOF // End of File
```

- Explanation: The `current_position` keeps track of where we are in the buffer. The essential character is the next character needed to form a valid token. When the buffer is empty, the lexer might need to refill the buffer or signal the end of input (EOF).